

Sri Lankan Speaker Verification (Sinhala / Tamil) — Project Analysis & Roadmap

Subject repo: voxceleb_trainer/ (fork: SL_ColvaiaAI, owner: dimuthuanuraj) **Analysis date:** 2026-05-14 **Scope:** (1) usability assessment for Sri Lankan speaker verification (SV) research, (2) accuracy-improvement plan for Sinhala and Tamil, (3) bug & engineering-debt audit.

1. What this repository actually is, today

A fork of Clova AI's [voxceleb_trainer](#), heavily extended by Anuraj (October-December 2025) for the **Sri Lankan Celebrity (SL_Celeb)** project. Despite the "SL_" branding, every config, training list, augmentation, and test pair shipped in the repo points at **English VoxCeleb1/VoxCeleb2** (5,991 speakers, 1.09M utterances). No Sinhala or Tamil data is present yet — the project is currently a **performance-optimised English-SV trainer** that is being shaped to serve as the *backbone* for Sri Lankan work.

Components in place

Category	Files
Training entry points	trainSpeakerNet.py , trainSpeakerNet_performance_updated.py , trainSpeakerNet_distillation.py
Model classes	SpeakerNet.py , SpeakerNet_performance_updated.py , SpeakerNet_distillation.py , DistillationWrapper.py
Dataset/IO	DatasetLoader.py , DatasetLoader_performance_updated.py
Architectures	ResNetSE34L/V2, VGGVox, RawNet3, NestedSpeakerNet (custom, unstable), LSTMAutoencoder (teacher), MLPMixerSpeaker (student, mel), MLPMixerSpeaker_RawWaveform (student, raw)
Losses	softmax, am/aam-softmax, ge2e, proto, angleproto, softmaxproto, triplet
Eval	tuneThreshold.py → EER, MinDCF
Optimisation	LRU cache, AMP/FP16, persistent workers, prefetch=3, FFT-based RIR convolution

Best reported result (researcher's logs)

- **Teacher** LSTM+AutoEncoder (3.87M params): **9.68% EER** on VoxCeleb1
- **MLP-Mixer V2 student** (2.66M params, cosine-similarity distillation, $\alpha=0.7$): **10.32% EER** — current SOTA in this repo
- **MLP-Mixer V2_Large_LowAlpha** (7.84M params, $\alpha=0.4$): **10.11% EER**
- ResNetSE34L baseline on mini-VoxCeleb1 (140 speakers): **15.48% EER**

All numbers are on English VoxCeleb1 test pairs; **none on Sinhala / Tamil**.

2. Using this repo for Sinhala / Tamil speaker verification

2.1 What carries over for free

The training/eval contract is **language-agnostic**:

- `train_list.txt`: `speaker_id relative/path/to.wav` (one per utterance)
- `test_list.txt`: `0|1 enrol.wav test.wav` (trial pairs, label optional in eval)
- Audio assumption: mono PCM, **16 kHz** (`max_audio = max_frames * 160 + 240` at [Dataset-Loader.py:29](#) hard-codes 10 ms hop and 25 ms window).
- All loss functions are speaker-classification or metric-learning losses; they do not encode any linguistic content.
- The MLP-Mixer V2 and LSTM+AE checkpoints are reusable as **multilingual initialisations** — speaker identity is largely language-independent at the embedding level.

So in principle you can:

1. Assemble a Sri Lankan speaker corpus.
2. Write `train_list.txt` / `test_list.txt` in the same format.
3. Drop in any existing config and start training or fine-tuning.

2.2 What does NOT carry over (this is where the real work is)

Issue	Why it matters for SL languages
No Sinhala/Tamil training data	VoxCeleb is English celebrity YouTube; phoneme inventory, prosody, breath/pause patterns differ. SL speakers also frequently code-switch (Sinhala↔English, Tamil↔English, sometimes all three).
No telephony / 8 kHz support	Many real SL deployment domains (call-centre, KYC, telco fraud) are 8 kHz μ -law. The data loader assumes 16 kHz and will silently mis-frame 8 kHz audio.
No VAD / silence handling	SL broadcast and field recordings have long silences, music beds, and overlapping talk. <code>loadWAV</code> uses <code>numpy.pad(..., 'wrap')</code> on short audio — wrapping silence repeats acoustic content unrealistically.
No language-aware evaluation	A single global threshold across Sinhala-only, Tamil-only, and code-switched trials will under-perform. Score distributions differ per language.
Augmentation tuned to MUSAN/RIR	These cover Western soundscapes. SL acoustic environments (tropical outdoor, three-wheeler, temple bells, call-centre comfort noise) are underrepresented.
Class count hard-coded	<code>nClasses</code> in every config is set to 5991/5994 (VoxCeleb2). Must change per Sri Lankan corpus.
Distillation teacher is English-only	The 9.68% EER LSTM+AE was trained on English data. Distilling Sinhala embeddings <i>from</i> it will leak English-dominated structure.

2.3 Recommended data sources to bootstrap

Source	Lang	Notes
OpenSLR SLR52	Sinhala	~185k utterances, ~70 speakers, TTS-style; good for <i>pretraining</i> but limited speaker count.
OpenSLR SLR65	Tamil	Crowd-sourced, multiple speakers.
OpenSLR SLR66	Tamil	Larger Tamil corpus.
Mozilla Common Voice (si, ta)	Sinhala/Tamil	Crowd-sourced; metadata includes speaker IDs.
FLEURS	si, ta among 102 langs	12 utterances/speaker; useful as cross-lingual test set.
Sri Lankan parliament Hansard recordings	Sinhala/Tamil/English	Public-domain, code-switched, multi-speaker, near-ideal ; needs diarisation and speaker labelling.
ITN/Rupavahini news archives	All three	Broadcast quality, identifiable presenters; copyright must be cleared.
Self-collected mobile recordings	Targeted	For domain-matched evaluation.

Minimum viable SL_Celeb corpus to start: ~200 speakers $\times$ ~20 utterances each, balanced Sinhala/Tamil, plus a held-out 100-speaker test set with both same-language and **code-switched** trial pairs.

2.4 Concrete usage recipe

Step 1 — corpus assembly

```
data/sl_celeb/  
├ wav/<speaker_id>/<utt_id>.wav      # mono, 16 kHz,  $\geq 2$  s after VAD  
├ train_list.txt                     # <speaker_id> <relpath>  
├ test_list.txt                      # <label> <enrol> <test>  
└ metadata.tsv                      # speaker_id language(si/ta/en/mix) gender age_bucket channel
```

Step 2 — fine-tune from the best English checkpoint Use configs/mlp_mixer_distillation_v2_large_low as the base, copy to configs/sl_celeb_finetune.yaml, and:

```
initial_model: exps/mlp_mixer_distillation_v2/model/model000000060.model  
train_list: data/sl_celeb/train_list.txt  
test_list: data/sl_celeb/test_list.txt  
train_path: data/sl_celeb/wav  
test_path: data/sl_celeb/wav  
nClasses: <your speaker count>  
lr: 0.0001                # 10 $\times$  lower for fine-tuning  
lr_decay: 0.98  
max_epoch: 60  
augment: true  
patience: 10
```

Step 3 — evaluate per-language Split test_list.txt into test_list_si.txt, test_list_ta.txt, test_list_cs.txt (code-switched), test_list_cross.txt (e.g., Sinhala enrol vs. Tamil test for the same speaker). Run --eval on each and compare EER/MinDCF.

3. How to improve accuracy on Sinhala and Tamil

These are ranked by **expected gain per engineering hour** for someone working alone on 2 $\times$ T4 GPUs.

3.1 Tier-1 wins (low effort, high impact)

1. **Cross-lingual fine-tuning**, not training from scratch. (Config knobs landed via [FEATURE-004](#) on 2026-05-19. Layer-wise LR decay added by [FEATURE-005](#) on 2026-05-19 — `llrd: true + llrd_decay: 0.9 + llrd_layer_pattern: ssl` make this a one-line YAML change, composes with `finetune`. The bare form (set `initial_model` + low lr) was always supported; [FEATURE-004/005](#) add selective-freeze, LR scaling, per-layer LR decay, and the typo-catching assertion. Empirical SL run still pending.)

The best English checkpoint already encodes speaker-invariant features. With $\leq 10k$ SL utterances, fine-tune with low LR; with $\geq 100k$, also unfreeze the SincConv front-end.

1. **Add a learnable language-aware front-end**. (Implementation landed via [FEATURE-001](#) on 2026-05-19 — both paths now selectable via config. Empirical SL evaluation still pending.)

The mel-spectrogram is fixed; Sinhala phoneme energy concentrates differently. Switch to `MLPMixerSpeaker_RawWaveform` so SincConv bands adapt to Sinhala/Tamil formant distributions (see [configs/language_aware_sincconv.yaml](#)), or replace the mel block with a frozen **WavLM-Base / XLS-R-300M / mHuBERT-147** encoder via the new `models/SSLFrontendSpeaker.py` (see [configs/language_aware_ssl_wavlm.yaml](#), [_xlsr.yaml](#), [_mhubert.yaml](#)). These multilingual SSL models cover Sinhala and Tamil and give 30–50% relative EER reduction on low-resource languages in the literature.

1. **Score normalisation (AS-Norm)**. (Implementation landed via [FEATURE-002](#) on 2026-05-19 — opt-in via `--as_norm`; reports raw VEER/MinDCF alongside the AS-Norm pair for diagnostics. Empirical SL validation still pending.)

The `MINDCF_IMPROVEMENT_GUIDE.md` flags this as missing. It typically yields **10–20% relative MinDCF reduction** at zero training cost. Build a cohort of ~ 500 SL speaker embeddings; normalise each trial score by cohort-mean/std.

1. **Per-language threshold calibration**. (Per-language evaluation landed via [FEATURE-003](#) on 2026-05-19 — opt-in via `--per_lang_test_lists 'si:path,ta:path,cs:path'`; reports per-language and pooled VEER/MinDCF/Threshold and composes with `--as_norm`. Platt logistic calibrator is documented as a follow-up. Empirical SL validation still pending.)

After training, fit a separate decision threshold for Sinhala-only, Tamil-only, and code-switched trial sets, or fit a simple logistic-regression calibrator (Platt) using language ID and raw score as inputs. Operational EER drops noticeably.

1. **Augmentation tuned to deployment channel**.

If your deployment is telephony, **simulate 8 kHz down/upsampling, μ -law companding, and codec artefacts (G.711, AMR-NB, Opus 8 kHz)** during training. The current MUSAN/RIR pipeline does *not* do any of this. Adding it usually gives 1–3% absolute EER on telephony evaluation.

3.2 Tier-2 (moderate effort, strong gains)

1. **Add ECAPA-TDNN**. (Implementation landed via [FEATURE-006](#) on 2026-05-19 — `models/ECAPA_TDNN.py` (Desplanques et al. 2020) + [configs/ecapa_tdn.yaml](#) + `ecapa` LLRD alias. Smoke-tested: 14.26M params (large), 5.80M (small at `channels: 512`), $[B, T] \rightarrow [B, n_{out}]$ finite outputs. Empirical baseline run still pending.)

Currently the strongest practical SV architecture (consistently 0.8–1.5% EER on VoxCeleb1-O). Not present in `models/`. Drop in a `Conv1d+SE+attentive-stat-pooling` implementation and add a config. This alone is likely worth 2–4% absolute EER on SL data versus `ResNetSE34L`.

1. **Multi-task language ID auxiliary head**. (Implementation landed via [FEATURE-007](#) on 2026-05-19 — `lang_aux: true + lang_aux_weight: 0.3 + lang_aux_label_file: <spk_lang_lookup.txt>` add `L_speaker + \lambda \cdot L_lang` during training only; eval paths untouched. Smoke-tested: forward returns finite loss with aux term, default-off path doesn't even instantiate the head. Per-utterance lookup is documented as a follow-up. Empirical SL run still pending.)

On the shared backbone add a small classifier predicting language (si/ta/en/mix). Adds $L_{\text{speaker}} + \lambda \cdot L_{\text{lang}}$. Forces embeddings to be informative about the speaker *given* the language — empirically reduces cross-lingual EER by 10–15% relative.

1. Speaker-augmented mixup of Sinhala and English/Tamil.

Take pairs of utterances from the *same* speaker in different languages and treat them as positive pairs in metric-learning losses. Even a few thousand such pairs from the Hansard corpus build code-switch robustness.

1. Re-distill with a multilingual teacher.

Replace the English LSTM+AE teacher with a public multilingual model (e.g., a fine-tuned WavLM-SV or ECAPA-TDNN trained on VoxCeleb2 + CN-Celeb + a SL subset). The current cosine-distillation pipeline in [SpeakerNet_distillation.py](#) needs only a new `teacher_model` and `teacher_checkpoint`.

1. Sub-centre AAM-Softmax.

Already discussed in [MINDCF_IMPROVEMENT_GUIDE.md](#); not yet implemented in [loss/aamsoftmax.py](#). Helps with noisy crowd-sourced data which Sinhala/Tamil sources will inevitably contain.

3.3 Tier-3 (research-grade)

1. **Domain adversarial training** for channel and language invariance (DANN-style head; reverses gradient on language/channel classifier). *(Implementation landed via [FEATURE-008](#) on 2026-05-19 — `GradientReversalFn` + `DANNHead` modules; enable per signal via `dann_lang: true` and/or `dann_channel: true` with per-speaker lookup files. Smoke-tested: GRL produces sign-flipped, scaled gradients on the encoder; default-off path doesn't instantiate the heads. NOTE: opposite objective to [FEATURE-007](#) on language — pick one, not both. Empirical SL run still pending; lambda ramp schedule is documented as a follow-up.)*
 2. **Self-supervised pretraining on unlabelled SL audio** — there are hundreds of hours of unlabelled Sinhala/Tamil podcast and broadcast audio. SSL pretrain a wav2vec-2 style encoder, then SV-finetune. *(Scoped as design-only via [FEATURE-009](#) on 2026-05-19 — deferred, no code yet. The bulk of the SSL benefit is already captured by [FEATURE-001](#) (off-the-shelf multilingual SSL); [FEATURE-009](#)'s continued-pretraining (CPT) path adds an estimated 5–15 % relative EER on top, but only when ≥ 500 h of unlabelled SL audio exists AND the [FEATURE-001](#) baseline has plateaued. Implementation is a one-day landing once those prerequisites are met; the design doc captures the architecture, hyper-parameter defaults, and pre-flight checklist.)*
 3. **Probabilistic Linear Discriminant Analysis (PLDA) scoring** on top of embeddings — historically standard in NIST SRE, still useful for short-utterance trials common in telephony. *(Implementation landed via [FEATURE-010](#) on 2026-05-19 — `plda.py` (simplified two-covariance PLDA + length-norm + LDA preprocessing) hooked into `SpeakerNet.evaluateFromList`; enable via `plda: true` + `plda_train_list: <path>`. Smoke-tested on synthetic embeddings: same-vs-different score gap +11.8 with 0% synthetic EER; save/load round-trips. Composes with `--as_norm`. Empirical SL benefit will surface most clearly under short-utterance / telephony deployment scenarios per §3.1 #5.)*
-

4. Bugs and engineering debt

I went through every file. Bugs are grouped by severity. Line numbers are accurate as of this analysis.

4.1 Critical (will silently corrupt experiments)

1. `trainSpeakerNet.py` — DDP spawn uses wrong kwarg.

[trainSpeakerNet.py:411](#) calls `mp.spawn(main_worker, n_procs=n_gpus, args=...)`. `torch.multiprocessing.spawn` accepts `nprocs`, not `n_procs`. Any `--distributed` run will raise `TypeError`. Fix: rename to `nprocs=`.

(trainSpeakerNet_performance_updated.py likely shares this bug; check before any multi-GPU run.)

1. ****trainSpeakerNet.py — current_threshold = result[2] is not a threshold.****

[trainSpeakerNet.py:241,291](#) — tuneThresholdFromScore returns (tunedThreshold, eer, fpr, fnr). result[2] is the **fpr array**, not a scalar threshold. Subsequent f'{current_threshold:f}' will raise TypeError (the _performance_updated script fixes this by recasting float(current_threshold) first — port that fix back to the original script, or always use the perf-updated trainer).

1. **SpeakerNet.py — TEER/TAcc displays as 0% when nPerSpeaker > 1.**

Documented at length in [2025-10-30.md](#). It's labelled "cosmetic", but it means the **training loop's own accuracy metric is wrong** whenever metric-learning batches are used. Validation EER is fine, but train-time monitoring is misleading and could mask convergence problems. Real fix: subsample labels with label = label[:nPerSpeaker] after the embedding-mean reshape — and verify before merging, since an earlier attempt at this was reverted.

1. **SpeakerNet.py:40 — hard .cuda() call.**

data = data.reshape(...).cuda() will crash on CPU-only machines and on the wrong GPU index in multi-GPU jobs. Replace with .to(self.gpu, non_blocking=True) and propagate self.gpu into WrappedModel/SpeakerNet.

1. **DatasetLoader.py:29 — 16 kHz hard-coded.**

max_audio = max_frames * 160 + 240 silently assumes 16 kHz. Loading 8 kHz telephony will produce half-length clips and break ASP pooling shapes. Fix: read sample_rate from soundfile.read, resample to a configured sample_rate parameter, and compute frame length from it.

1. **DatasetLoader.py:38 — numpy.pad(audio, ..., 'wrap') for short audio.**

Wraps audio onto itself, fabricating phantom repetitions. Real fix: mode='constant' (silence pad) with energy-based VAD warning, or repeat *with* small Gaussian noise to avoid creating perfectly periodic features that the model can learn to overfit.

1. **MLPMixerSpeaker_RawWaveform.py — SincConv buffer placement.**

Sinc filter buffers are moved to device on every forward pass; this is both slow and a hidden bug if running on multi-GPU because they end up on the wrong device. Move to register_buffer(..., persistent=False) in __init__.

1. **RawNetBasicBlock.py:100 — nn.MaxPool1d(pool) if pool else False.**

When pool is falsy this returns the literal False, then later code tries to call it; will crash unless every config sets a truthy pool size. Use nn.Identity() instead of False.

1. **models/NestedSpeakerNet.py — known NaN explosions** documented across [2025-12-29-nested-learning-experiment.md](#). Three attempts, three crashes. The architecture as designed is not viable for variable-length audio. Recommend retiring the file or moving it to models/experimental/ with a README header explaining it does not converge.

4.2 Important (degrade quality or reproducibility)

1. **requirements.txt has loose pins** (torch>=1.7.0) yet the code uses torch.amp.autocast('cuda', ...) (PyTorch ≥2.1), torch.inference_mode, set_to_none=True, GradScaler features. Real lock-file or pin to torch>=2.1,<3.

1. **✓ analyze_nan_debug.py and NaN_DEBUGGING_GUIDE.md are 0-byte placeholders.** (Closed by [BUGFIX-012](#) — filled with a real diagnostic script and guide grounded in the BUGFIX-010 / NestedSpeakerNet incident.)

1. **lists/ is empty of SL data**; only the legacy VoxCeleb files (fileparts.txt, files.txt, augment.txt) exist. dataprep.py is therefore unusable for Sri Lankan data; needs a parallel sl_dataprep.py. (Closed by [FEATURE-011](#) on 2026-05-22 — [tools/sl_dataprep.py](#) walks a <root>/<lang>/<spk>/<utt>.wav tree and emits all nine list/lookup/cohort files in one invocation. Smoke-tested on a synthetic 100-speaker corpus. The end-to-end protocol is documented in [RUN_GUIDE.md](#).)

1. ✓ **Configs point at private mount paths** like `/mnt/ricproject3/...`, `/mnt/ricproject2/...`. (Closed by [BUGFIX-013](#) — `env-var` substitution in YAML loader; configs use `#{SL_SPV_DATA_ROOT}` etc.; setup documented in `paths.env.example`.)
1. ✓ **models/ResNetSE34L.py and VGGVox.py hard-code n_mels**. (Closed by [BUGFIX-014](#) — `VGGVox.py` now honours `n_mels` and raises `ValueError` for architecture-incompatible values; `ResNetSE34L.py` was audited and already honours the parameter, so the original bug report was inaccurate for that file.)
1. ✓ **RawNet3.py:49,88 — print() and in-place modification**. (Closed by [BUGFIX-015](#) — debug print removed; `s[s < 0.001] = 0.001` replaced with chained `.clamp(min=0.001)`, matching the existing idiom on lines 110 / 124 of the same file.)
1. ✓ **Augmentation hard-codes 5 fixed choices**. (Closed by [BUGFIX-016](#) — `augment_chain` config block accepted in dict / list / JSON-string forms, with validation; default behaviour is the legacy uniform 0.2 each.)
1. ✓ **No deterministic mode toggle**. (Closed by [BUGFIX-017](#) — `--deterministic` flag added to all three trainers; seeds `random / numpy / torch`, disables `cupy.benchmark` and `TF32`, enables `torch.use_deterministic_algorithms(True, warn_only=True)`.)
1. ✓ **evaluateFromList loads the whole feature dict into a single dict on rank 0**. (Closed by [BUGFIX-018](#) — opt-in `--eval_streaming` flag writes per-file embeddings to `<save_path>/eval_feats_tmp/` and lazy-loads with an LRU cache; peak memory becomes $O(\text{cache_size} \times \text{per-embedding})$ instead of $O(N_{\text{files}} \times \text{per-embedding})$.)
1. ✓ **SpeakerNet.py:248 — torch.load(...) without weights_only=True**. (Closed by [BUGFIX-019](#) — repo-wide audit found 8 `torch.load` sites (not 1); all now carry `weights_only=True`. No checkpoint this repo wrote is affected since `saveParameters` already produces pure state_dicts.)
1. ✓ **tuneThreshold.py EER returns max(fpr[idxE], fnr[idxE]) * 100**. (Closed by [BUGFIX-020](#) — module docstring rewritten with the closed-form discrepancy $|FPR-FNR|/2$ and granularity bound $\sim 1/(2N)$; return tuple grew to 6 elements with `eer_average` at index 5; trainer banners now print both `VEER` and `VEER_avg`.)

4.3 Minor / polish

1. ✓ `pdb` imports left in production. (Closed by [BUGFIX-021](#) — repo-wide audit found 10 files with the issue, not the 3 named; all swept.)
2. ✓ Inconsistent variable casing (`nClasses`, `nDataLoaderThread`, `nOut` vs. `lr_decay`). (Closed by [BUGFIX-022](#) — `snake_case` aliases accepted on CLI (`argparse` multi-name) and in YAML (key-normalization pass); canonical names stay camelCase so all 19 existing configs and every internal `args.nClasses` reference still works.)
3. ✓ `dataprep.py` MD5 mismatch raises `Warning` as exception. (Closed by [BUGFIX-023](#) — both sites now raise `ValueError(...)` with expected/observed MD5 in the message, matching the existing `ValueError` convention used by the download/conversion failure paths.)
4. ✓ `analyze_performance.py`, `benchmark_performance.py`, `quick_optimize.py` overlap heavily; consolidate. (Closed by [BUGFIX-024](#) — `analyze_performance.py` and `quick_optimize.py` moved to `scripts/archive/` with a policy `README`; `benchmark_performance.py` survives at root with refreshed docstring.)
5. ✓ Each model file duplicates `PreEmphasis`, `mel` transforms, `InstanceNorm`. (Closed by [BUGFIX-025](#) — new `models/_frontend.py` owns the canonical `PreEmphasis(squeeze=...)` and a `make_mel_frontend(sample_rate, n_mels, pre_emphasis)` factory; 4 `mel` models migrated; legacy `utils.PreEmphasis` and `models.RawNetBasicBlock.PreEmphasis` preserved as thin subclasses; `state_dict` keys unchanged.)
6. ✓ The 12 experiment folders under `exps/` mostly contain only `logs/` and `result/`, no model checkpoints. (Closed by [BUGFIX-026](#) — new `exps/README.md` documents the four-tier retention policy, the never-delete list, resume semantics, and §2.4's action plan for the 11 currently-unresumable directories. Documentation only; no destructive action performed.)

5. Proposed plan of attack (12-week timeline, single researcher, 2× T4)

Week	Goal	Deliverable
1	Fix critical bugs §4.1 #1-#8	A clean main, passing test_data_loader.py
1	Add sample_rate + telephony resampling in DatasetLoader	New helper load_audio_resampled
2	Assemble Pilot-SL corpus: 50 Sinhala + 50 Tamil speakers, OpenSLR + Hansard	data/sl_celeb_pilot/ ready
2	Add sl_dataprep.py and per-language test_list_*.txt	Reusable data tooling
3	Implement AS-Norm + per-language calibration	score_norm.py; baseline EER table
4	Cross-lingual fine-tune of MLP-Mixer V2 on pilot	First Sinhala/Tamil EER number
5	Add ECAPA-TDNN to models/; train baseline	New SOTA candidate
6	Add WavLM-Base front-end variant	Test SSL benefit
7	Telephony augmentation (8 kHz, μ -law, codecs)	Telephony eval set
8	Multi-task language-ID head	Code-switched EER reduction
9	Scale corpus to 200+ speakers, full Hansard ingest	Updated leaderboard
10	Sub-centre AAM-Softmax	Loss ablation
11	Multilingual teacher re-distillation	Compressed SL model
12	Paper-grade ablation + reproducibility (deterministic seeds, cards)	Camera-ready experiments

6. TL;DR

- The repo is a **well-instrumented English speaker-verification trainer** with novel MLP-Mixer + distillation contributions (10.32% EER on VoxCeleb1) — it is not yet a Sri Lankan SV system.
- For Sinhala/Tamil work, treat it as a **starting backbone**: fix the bugs in §4.1, build a Sri Lankan corpus with proper per-language and code-switched test pairs, and **fine-tune** the existing MLP-Mixer or LSTM+AE checkpoints rather than training from scratch.
- The two biggest accuracy levers for Sinhala/Tamil are (a) **multilingual SSL front-end** (WavLM/XLS-R/mHuBERT) and (b) **score normalisation + per-language calibration**. Both are missing and both are cheap.
- The biggest engineering risks are the **silent-bug cluster** in §4.1 (#1, #2, #3 especially) — fix these before any paper-grade run.

7. Audit close-out: BUGFIX retrospective (added 2026-05-19)

The §4 bug audit was worked through end-to-end in a 5-day pass (2026-05-14 → 2026-05-19). Each item was tracked as a BUGFIX-NNN under [docs/bugfixes/](#) with a uniform 8-section template

(Problem / Fix / Verification / Backwards-compatible / Out-of-scope / Rollback / Cross-refs / Authorship). The §4 entries above carry inline ✓ marks for the items closed; some of the earlier ✓ marks were not back-filled into the §4 list itself — this section is the authoritative reference.

7.1 Full close-out table (26 fixes)

BUGFIX	§4 item	Severity	One-line outcome
001	§4.1 #1	Critical	<code>mp.spawn(n_procs=...) → nprocs=...; multi-GPU runs no longer TypeError at spawn.</code>
002	§4.1 #2	Critical	<code>tuneThresholdFromScore</code> returns the EER-point threshold as the 5th element; trainers consume <code>result[4]</code> .
003	§4.1 #3	Critical	Dispatch metric vs. softmax losses by <code>expects_grouped_input</code> ; TEER/TAcc now accurate for <code>nPerSpeaker > 1</code> .
004	§4.1 #4	Critical	<code>_resolve_device()</code> helper in each trainer; 16 <code>.cuda(...)</code> sites → <code>.to(self.device, ...)</code> . Runs on CPU.
005	§4.1 #5	Critical	<code>_frame_to_samples + _resample_if_needed</code> ; <code>--sample_rate</code> flag; 16 kHz default preserved.
006	§4.1 #5 follow-up	Critical	Threaded <code>sample_rate=</code> into every model's <code>MelSpectrogram</code> call (8 model files).
007	§4.1 #6	Critical	<code>numpy.pad(..., 'wrap')</code> → <code>silence-pad + Gaussian dither (~-80 dBFS)</code> ; one-time <code>stderr</code> warning.
008	§4.1 #7	Critical	<code>SincConv</code> window / lookup via <code>register_buffer(..., persistent=False)</code> .
009	§4.1 #8	Critical	<code>MaxPool1d(pool)</code> if <code>pool</code> else <code>False</code> → <code>nn.Identity()</code> ; latent crash defused.
010	§4.1 #9	Critical	<code>NestedSpeakerNet</code> moved to <code>models/experimental/</code> with policy README; configs use <code>experimental.NestedSpeakerNet</code> .
011	§4.2 #10	Important	<code>torch>=2.1,<3</code> and <code>friends</code> ; <code>numpy<3</code> (NumPy 2.x supported); all major-bounded.
012	§4.2 #11	Important	<code>Real analyze_nan_debug.py + NaN_DEBUGGING_GUIDE.md</code> grounded in the BUGFIX-010 incident.
013	§4.2 #13	Important	<code>\${SL_SPV_*}</code> expansion in YAML loader; 16 configs rewritten; <code>paths.env.example</code> .
014	§4.2 #14	Important	VGGVox honours <code>n_mels</code> and raises on incompatible values; report was inaccurate for <code>ResNet6524</code> .

7.2 Audit coverage by severity

Severity	Audit items	Closed	Open	Open items
§4.1 Critical	9	9	0	—
§4.2 Important	11	10	1	#12 (lists/empty — Sri Lankan dataprep feature build, not a bug)
§4.3 Minor / polish	6	6	0	—
Total	26	25	1	

The one open item (§4.2 #12) is genuinely a feature build, not a bug. It is the start of §5's Week 2 work, not the tail of the engineering pass.

7.3 Where the audit was inaccurate (recorded honestly per fix)

Four §4 items had bug-report claims that turned out to be partly or fully wrong on close inspection. Each corresponding BUGFIX doc flags the deviation in its §1 rather than silently fixing the wrong thing:

Audit item	Reported	Actual finding	Doc §
§4.1 #8 (MaxPool1d if pool else False)	"will crash"	Currently <i>latent</i> (guarded by <code>if self.mp:</code>); still fixed for safety	BUGFIX-009 §1.2
§4.2 #10 (<code>torch.amp.autocast('cuda2,1 API ...')</code>)	Code uses PyTorch ≥ 2.1 API	Code uses older <code>torch.cuda.amp.autocast()</code> form; pin still justified	BUGFIX-011 §1.2
§4.2 #14 (ResNetSE34L hard-codes <code>n_mels</code>)	Both VGGVox and ResNetSE34L	Only VGGVox does; ResNetSE34L already honoured the parameter	BUGFIX-014 §1.2
§4.2 #19 (<code>SpeakerNet.py:248</code>)	One site	Eight sites total across five files	BUGFIX-019 §1.1

Recording these honestly in the BUGFIX trail is the project's audit-of-the-audit. Future hand-offs need this signal — quoting the original §4 wording verbatim in a paper or grant application would, in four cases, repeat an inaccuracy.

7.4 Process artefacts the audit produced as side effects

In addition to the 26 fix docs, the audit pass produced three operational artefacts that did not exist before:

Artefact	Purpose	Created by
<code>paths.env.example</code>	Three-variable contract for portable configs	BUGFIX-013
<code>scripts/archive/</code> + <code>README</code>	Quarantine directory for obsolete tooling	BUGFIX-024
<code>models/experimental/</code> + <code>README</code>	Quarantine directory for non-convergent models	BUGFIX-010
<code>exps/README.md</code>	Retention policy for experiment artefacts	BUGFIX-026
<code>models/_frontend.py</code>	Canonical PreEmphasis + <code>make_mel_frontend</code> factory	BUGFIX-025

The `<dir>/README.md` pattern (set down by BUGFIX-010, reused by BUGFIX-024 and BUGFIX-026) is now a documented project convention for "directory whose contents differ from naive expectations".

8. Where you are on the §5 12-week plan (added 2026-05-19)

The 12-week timeline in §5 above was written on 2026-05-14. Five days later, this is the actual state. The short version: **the engineering hygiene is done; the research has not started yet.**

8.1 Annotated week-by-week status

Week	\$5 goal	Status as of 2026-05-19	Notes
1	Fix critical bugs \$4.1 #1-#8	✓ Complete	All eight closed (BUGFIX-001..009). #9 also closed (BUGFIX-010 — quarantine).
1	Add sample_rate + telephony resampling in DatasetLoader	✓ Complete	BUGFIX-005 (loader-side) + BUGFIX-006 (model-side threading). Resampling implemented via <code>scipy.signal.resample_poly</code> .
— scope additions that \$5 did not budget	—	✓ \$4.2 #10, #11, #13-#20 closed (10 fixes); \$4.3 #21-#26 closed (6 fixes).	The audit pass went well beyond \$5's Week 1 scope.
2	Assemble Pilot-SL corpus: 50 Sinhala + 50 Tamil speakers, OpenSLR + Hansard	☐ Not started	Requires a data-collection workstream, copyright clearance for Hansard / news archives, and probably a non-engineering collaborator.
2	Add <code>sl_dataprep.py</code> and per-language <code>test_list_*.txt</code>	✓ Closed by FEATURE-011 (2026-05-22)	Script ships at <code>tools/sl_dataprep.py</code> ; the full experimental protocol consuming it lives at <code>RUN_GUIDE.md</code> . \$4.2 audit table is now 100 % closed.
3	Implement AS-Norm + per-language calibration	✓ Both done (FEATURE-002 + FEATURE-003, 2026-05-19)	\$3.1 #3 / #4. AS-Norm ships behind <code>--as_norm</code> ; per-language eval ships behind <code>--per_lang_test_lists</code> . Platt logistic calibrator left as documented follow-up. Empirical SL validation pending.
4	Cross-lingual fine-tune of MLP-Mixer V2 on pilot	☐ Not started	Blocked on Week 2 (corpus).
5	Add ECAPA-TDNN to <code>models/</code> ; train baseline	☐ Implementation done (FEATURE-006, 2026-05-19), training not started	\$3.2 #6. Model + config + LLRD alias shipped; the actual training run is a one-line YAML invocation away.
6	Add WavLM-Base front-end variant	☐ Not started	\$3.1 #2. Same comment as ECAPA — can be prototyped on English in parallel.
7	Telephony augmentation (8 kHz, μ -law, codecs)	☐ Not started	The <i>primitive</i> (resampling) is now in place (BUGFIX-005); the <i>augmentation policy</i> (random downsample + μ -law during training) is not.
8	Multi-task language-ID head	☐ Implementation done (FEATURE-007, 2026-05-19), training run blocked on Weeks 2 + 4	\$3.2 #7. Aux head + lookup loader + CLI/YAML knobs all shipped and smoke tested; estimate

8.2 Honest assessment — are you on the correct path?

Yes for engineering, no for research delivery.

- The path so far has been: *close every reachable engineering

debt before starting research work.* That is a defensible and rare discipline — most projects accumulate debt and pay compounding interest. Twenty-six closed audit items in five days is sustained, focused work.

- BUT: the original \$5 plan budgeted **one week** for Week-1

engineering and assumed Week 2 (corpus + dataprep) would be running by 2026-05-21. As of 2026-05-19, you are five days in and have done ~5 weeks' worth of engineering — none of it on the \$5 plan past the first row, and **zero research output**. No SL data, no SL EER number, no fine-tune run, none of the \$3 Tier-1 wins prototyped.

- The risk this creates: the project becomes "the world's

best-engineered English speaker-verification trainer that never delivered Sinhala / Tamil results". Engineering completeness is necessary but not sufficient. The next pivot needs to be hard: from BUGFIX-NNN mode to corpus + fine-tune + EER number mode.

The \$1 framing was already explicit on this: *"Despite the 'SL_' branding, every config, training list, augmentation, and test pair shipped in the repo points at English VoxCeleb1/VoxCeleb2"*. Five days of engineering hygiene has not changed that observation. The trainer is still pointed at English; only the surface is different.

8.3 Where the \$5 plan still applies vs. needs updating

Plan items that survive unchanged:

- Weeks 2, 4, 7, 8, 9, 11 (all corpus / fine-tune / telephony /

language-ID / distillation work) — none of the engineering work changes the design of these steps.

- Week 12 — the BUGFIX trail itself is the reproducibility

infrastructure \$5 #12 imagined; less work remains here than originally planned.

Plan items the audit work makes cheaper or easier:

- **Week 3 (AS-Norm)** — can use BUGFIX-018's streaming

infrastructure as the embedding-store backend, instead of building one from scratch.

- **Week 7 (telephony augmentation)** — the `--sample_rate`

argument and `_resample_if_needed` helper from BUGFIX-005 are the substrate. Adding μ -law + bandwidth augmentation is a ~50-line `AugmentWAV.telephony(audio)` method now, not a weekend.

- **Week 12 (reproducibility)** — `--deterministic` (BUGFIX-017)

is the strict-RNG mode the paper-grade ablation depends on.

Plan items the audit work makes *harder* — none. The audit was strictly additive.

8.4 Implicit assumptions in \$5 that are still untested

- That a useful Sri Lankan pilot corpus *can* be assembled in

one week from OpenSLR + Hansard. Hansard copyright clearance, speaker labelling, and gender / language balance are open questions that engineering work doesn't address.

- That fine-tuning the existing MLP-Mixer V2 checkpoint on a

small SL corpus actually transfers. The \$3.2 hypothesis is plausible (speaker identity is largely language-invariant at the embedding level) but unverified for *these* checkpoints on *these* languages.

- That telephony deployment is a real near-term requirement. If

it isn't, Week 7's effort drops out of the plan.

These are research questions, not engineering ones. The audit pass cannot answer them.

9. Recommended next moves: pivoting to research (added 2026-05-19)

9.1 The hard pivot

Stop adding BUGFIX-NNN docs unless a critical bug appears in the course of research work. The engineering audit is closed enough that further polish is diminishing returns relative to the absence of any SL result. Concretely: **don't open §4.3 #27+ unless the issue is blocking a research deliverable.**

9.2 The next two weeks of concrete actions (priority-ordered)

#	Action	Outcome	Blocking?
1	Decide whether telephony / 8 kHz is in scope for the SL Pilot.	One-line decision: yes → keep Week 7 in the plan, no → drop it.	Blocks the corpus design (telephony requires explicit 8 kHz audio sources).
2	Pick a corpus floor: OpenSLR SLR52 (Sinhala) + SLR65/66 (Tamil) as the minimum-viable Pilot. Skip Hansard for now (copyright + diarisation are not 1-week problems).	A train list with ≥ 50 Sinhala + ≥ 50 Tamil speakers, mono 16 kHz, ≥ 2 s per utterance.	Blocks all of §3 / §5 Weeks 2–4.
3	Write <code>sl_dataprep.py</code> against that corpus. Closes §4.2 #12 properly (with real data, not as scaffolding).	A reusable corpus-prep script.	Blocks reproducibility of any SL result. <i>Shipped 2026-05-22 as <code>tools/sl_dataprep.py</code> (FEATURE-011).</i>
4	Establish a per-language SL test list: ≥ 1 k Sinhala-only pairs, ≥ 1 k Tamil-only pairs, ≥ 200 cross-language pairs (Sinhala enrol vs. Tamil test for same speaker if available, else cross-speaker as the contrast).	Three <code>test_list_*.txt</code> files.	Blocks all SL EER numbers.
5	Fine-tune <code>configs/mlp_mixer_distill</code> with <code>initial_model: <best English checkpoint></code> on the Pilot, low LR ($1e-4$), 60 epochs.	First SL EER number at the project's history page.	This is the binary deliverable. Until this number exists, the project is still "English SV trainer". <i>FEATURE-004 + FEATURE-005 (both 2026-05-19) make this a one-line finetune: <code>true + finetune_freeze: [frontend] + optional llrd: true toggle.</code></i>
6	Apply AS-Norm + per-language threshold calibration to that result.	Second SL EER number, expected 10–20% relative MinDCF reduction (§3.1 #3 / #4).	Cheap once #5 exists. <i>AS-Norm shipped 2026-05-19 (FEATURE-002); per-language eval shipped 2026-05-19 (FEATURE-003). Both are now flag-only re-runs.</i>

These six actions plug into §5 Weeks 2–4 cleanly and produce the first three deliverables of the original plan in roughly a week each — *if* the corpus questions in actions 1–2 don't expand into their own multi-week workstreams.

9.3 Parallel work that does not need SL data

These can proceed in parallel with actions 1–4 above and produce research outputs even before any SL data exists:

- **ECAPA-TDNN baseline on mini-VoxCeleb1.** §3.2 #6 / §5 Week 5.

Self-contained model addition; gives a stronger English baseline to fine-tune from later.

- **WavLM-Base front-end prototype on mini-VoxCeleb1.** §3.1 #2 /

§5 Week 6. Establishes the SSL plumbing before the SL data arrives so the fine-tune in action 5 above can use it directly.

- **Sub-centre AAM-Softmax loss experiment on mini-VoxCeleb1.**

§3.2 #10 / §5 Week 10. Self-contained loss change; one config + one loss file edit.

Any one of these three is worth more research-progress than another six BUGFIX docs would be at this point.

9.4 What to escalate to non-engineering collaborators

Things that need decisions from outside the engineering surface:

- **Hansard / Rupavahini copyright clearance.** Cannot be solved by

the engineer; needs a legal / institutional contact.

- **Sri Lankan dataset sponsorship / annotation budget.** Even

200 speakers $\times$ 20 utterances is $\sim$ 4000 labelled utterances; if it's not crowd-sourcing from existing public corpora, someone needs to pay for it.

- **Deployment scoping.** Whether the eventual product is

call-centre fraud detection, KYC, broadcast indexing, or something else changes the priority order of telephony work, long-utterance support, and code-switch handling.

The §5 plan implicitly assumes the engineer can answer these alone. They cannot.

10. Open decisions and known risks (added 2026-05-19)

10.1 Decisions the user still owes

#	Decision	Default if you don't choose	Cost of deferring
1	Is telephony / 8 kHz a Pilot-stage requirement?	Yes (the BUGFIX-005 primitive is in place; doing nothing keeps the option open)	Low — easy to add later.
2	What to do with the 11 unresumable exps/ directories?	Keep (no policy in this audit deletes them)	Negligible — they total $\sim$ 5 MB.
3	Whether to delete analyze_performance.py and quick_optimize.py outright (vs. keeping them in scripts/archive/)?	Keep in archive	None.
4	Whether NestedSpeakerNet deserves a fourth stabilisation attempt or permanent deletion?	Keep quarantined	None.
5	Whether to publish the BUGFIX trail as a methodology paper alongside the SL_SPV result?	Don't — focus on the SV result	Recoverable later.

None of these block research work. They are all “could be answered in a 30-second message” decisions that will accumulate as not-decided if left.

10.2 Known risks the BUGFIX pass did NOT remove

The audit closed audit-flagged issues. It did not address the following project-level risks, which are still live:

- **No SL data exists.** This is the single largest risk to delivery and is unchanged by any engineering work.

- **No SL EER number exists.** Therefore there is no baseline against which to measure any §3 improvement.

- **No external review of the engineering work.** Twenty-six

BUGFIX docs were authored in five days by one engineer + Claude. The reasoning in each doc is documented but has not been independently checked.

- **The MLP-Mixer V2 “10.32% EER” headline number** in §1 is

reported but not independently replicated. The --deterministic flag (BUGFIX-017) is new; the headline run predates it.

- ****The §3 Tier-1 #2 hypothesis (“WavLM/XLS-R/mHuBERT gives**

30-50% relative EER reduction on low-resource languages”)** is a literature claim, not something the project has verified for these specific languages or this specific backbone. Worth treating as a *prediction to test*, not a *given*.

10.3 Things that would invalidate the §5 plan

The plan is robust to many likely shocks. Things that would genuinely force a re-plan:

- **No Sri Lankan public corpus is usable.** Unlikely (OpenSLR

Sinhala / Tamil are usable today) but worth confirming on contact.

- ****Fine-tuning from the English MLP-Mixer V2 checkpoint does**

not transfer.** Possible — would force a from-scratch SL training run, which on 2× T4 GPUs and ~200 speakers may not reach VoxCeleb-quality EER within 12 weeks.

- **The deployment requirement changes mid-project** (e.g., from

broadcast to telephony, or from speaker verification to speaker identification / diarisation). The current architectures all do SV; changing the task changes the model family.

10.4 Bottom line

You are on the right path *for engineering*. You are at the **starting line** for research. Five days of audit work has not changed the §1 observation that the repo is an English speaker-verification trainer with no Sri Lankan content. The next action that matters is producing a first SL EER number — every hour spent on §4.3 #27 or §4.4 #N rather than corpus + fine-tune defers that delivery by an hour.

The §5 plan as written remains viable. Treat 2026-05-20 onward as **Week 2** of the original timeline, not as a deferred continuation of Week 1.